
Documentación Técnica - Sistema de Gestión de Condominio

Curso: Desarrollo Web y Móvil

Grupo 2 Fecha: 13 de Noviembre, 2025

1. Modelo de Datos

1.1 Diagrama Relacional

El sistema implementa un modelo de datos relacional normalizado con **11 entidades principales**:

Condominio (1) — (N) Vivienda
 | (N) EspacioComun
 └ (N) Anuncio

Vivienda (1) — (N) ResidenteVivienda
 | (N) GastoComun
 └ (N) Multa

Usuario (1) — (N) ResidenteVivienda
 | (N) Reserva
 | (N) Pago
 └ (N) Anuncio

EspacioComun (1) — (N) Reserva

GastoComun (1) — (N) Pago

1.2 Entidades Principales

Tabla	Descripción	Campos Clave
usuarios	Gestión de usuarios del sistema	email (único), password_hash, rol, is_active
condominios	Información de condominios	nombre (único), direccion
viviendas	Unidades habitacionales	numero_vivienda, cargo_fijo_uf, unique(condominio_id, numero_vivienda)
residentes_viviendas	Relación muchos-a-muchos Usuario-Vivienda	Clave compuesta: (usuario_id, vivienda_id)
gastos_comunes	Gastos mensuales por vivienda	mes, ano, monto_total, estado, unique(vivienda_id, mes, ano)
multas	Multas aplicadas a viviendas	monto, descripcion, fecha_aplicada
espacios_comunes	Espacios reservables	nombre, requiere_pago
reservas	Reservas de espacios	fecha_hora_inicio, fecha_hora_fin, google_event_id
pagos	Registro de pagos realizados	monto_pagado, metodo_pago, fecha_pago
anuncios	Comunicados y avisos	titulo, contenido, tipo, fecha_publicacion

1.3 Características del Modelo

- **Integridad Referencial:** Uso de ForeignKey con políticas CASCADE y RESTRICT según lógica de negocio
- **Restricciones:** UniqueConstraint, CheckConstraint para validación de datos
- **Índices:** Indexación en claves foráneas y campos de búsqueda frecuente
- **Auditoría:** Campos created_at y updated_at con timestamps automáticos

- **Normalización:** Forma normal 3NF, evitando redundancia de datos
-

2. Endpoints del Backend

2.1 Autenticación (/api/v1/auth)

Método	Endpoint	Descripción	Autenticación
POST	/register	Registro de nuevos usuarios	No
POST	/login	Inicio de sesión (retorna JWT)	No
GET	/me	Información del usuario actual	Sí (JWT)
PUT	/update-password	Cambio de contraseña	Sí (JWT)

Seguridad: Las contraseñas se encriptan con **bcrypt**. Los tokens JWT tienen expiración configurable (120 min en producción).

2.2 Gestión de Recursos

Recurso	Endpoints	Operaciones CRUD
Condominios	/api/v1/condominios	GET (list, detail), POST, PUT, DELETE
Viviendas	/api/v1/viviendas	GET (list, detail), POST, PUT, DELETE
Residentes	/api/v1/residentes	GET (list), POST (asignar), DELETE (desasignar)
Usuarios	/api/v1/usuarios	GET (list, detail), POST, PUT, DELETE
Gastos Comunes	/api/v1/gastos	GET (list, detail, by vivienda), POST, PUT, DELETE
Multas	/api/v1/multas	GET (list, detail), POST, PUT, DELETE
Espacios Comunes	/api/v1/espacios	GET (list, detail), POST, PUT, DELETE
Reservas	/api/v1/reservas	GET (list, my-reservas, disponibilidad), POST, DELETE

Recurso	Endpoints	Operaciones CRUD
Pagos	/api/v1/pagos	GET (list, by vivienda), POST, PUT
Anuncios	/api/v1/anuncios	GET (list, detail), POST, PUT, DELETE

2.3 Endpoints Especializados

- **Dashboard:** /api/v1/dashboard/stats - Estadísticas agregadas del sistema
- **Morosidad:** /api/v1/morosidad - Viviendas con pagos pendientes
- **Perfil:** /api/v1/perfil - Gestión de perfil de usuario
- **Disponibilidad:** /api/v1/reservas/disponibilidad - Consulta de horarios disponibles para reservas

Documentación Interactiva: Disponible en /docs (Swagger UI) y /redoc (ReDoc)

3. Decisiones Técnicas

3.1 Stack Tecnológico

Componente	Tecnología	Justificación
Framework Backend	FastAPI	Alta performance, tipado estático, documentación automática
ORM	SQLAlchemy 2.0	Estándar de facto en Python, soporte robusto para relaciones
Migraciones	Alembic	Versionamiento de esquema, integración nativa con SQLAlchemy
Base de Datos	MySQL 8.0	Requisito del proyecto, soporte empresarial, ampliamente utilizado
Autenticación	JWT + bcrypt	Stateless, escalable, seguro para aplicaciones web modernas
Containerización	Docker + Docker Compose	Portabilidad, facilidad de despliegue, ambientes consistentes

3.2 Arquitectura del Backend

backend/

```
|— app/
|   |— api/v1/           # Endpoints versionados
|   |   |— router.py     # Agregación de rutas
|   |   |   |— routes/   # Controladores por recurso
|   |   |— core/         # Lógica de negocio central
|   |   |— auth.py       # Autenticación JWT
|   |   |— security.py   # Encriptación bcrypt
|   |   |   |— config.py  # Configuración por variables de
entorno
|   |   |— db/           # Capa de base de datos
|   |   |   |— session.py # Motor SQLAlchemy
|   |   |   |— deps.py    # Inyección de dependencias
|   |   |— models/       # Modelos SQLAlchemy
|   |   |— schemas/      # Schemas Pydantic (validación)
|   |   |— services/     # Servicios externos (Google
Calendar)
|— alembic/              # Migraciones de BD
|— main.py               # Punto de entrada FastAPI
```

Patrón arquitectónico: Separación de responsabilidades (modelos, schemas, rutas, servicios)

3.3 Seguridad Implementada

1. **Hashing de contraseñas:** bcrypt con salt automático
2. **Tokens JWT:** Firma HMAC-SHA256, expiración configurable
3. **Middleware CORS:** Configurado para frontend en <http://localhost:5173>
4. **Validación de entrada:** Pydantic schemas con tipos estrictos

5. **Protección de endpoints:** Decorador `@get_current_active_user` para rutas protegidas
6. **Variables de entorno:** Secretos y configuración fuera del código fuente

3.4 Gestión de Migraciones

Alembic se configuró para:

- Autogeneración de migraciones desde modelos SQLAlchemy
- Versionamiento incremental del esquema
- Migración inicial: `20241112_000001_initial_schema.py` (11 tablas)
- Comando de ejecución: `alembic upgrade head`

3.5 Dockerización

docker-compose.yml orquesta 2 servicios:

1. **db (MySQL 8.0)**

- Puerto expuesto: 3306
- Volumen persistente para datos
- Healthcheck para garantizar disponibilidad
- Inicialización automática con `condominio_db.sql`

2. **backend (FastAPI)**

- Puerto expuesto: 8000
- Depende de db (espera healthcheck)
- Variables de entorno inyectadas
- Hot-reload en desarrollo (volúmenes montados)

Comando de despliegue: `docker-compose up -d`

3.6 Integración con Google Calendar

- **Opcional:** Para sincronización de reservas con calendarios externos
- **Implementación:** Servicio `google_calendar_service.py`
- **Configuración:** API Keys y Calendar IDs por espacio común

- **Estado actual:** Preparado para activación mediante variables de entorno

3.7 Control de Roles

Sistema de roles implementado:

- **Residente:** Acceso básico (perfil, reservas, pagos propios)
- **Conserje:** Gestión operativa (reservas, anuncios)
- **Directiva:** Aprobaciones y reportes
- **Administrador:** Control total del sistema
- **Super Admin:** Gestión multi-condominio

Validación de permisos a nivel de endpoint mediante decoradores de FastAPI.

4. Integración Frontend-Backend

4.1 Tecnologías Frontend

- **Framework:** React 18 + Vite
- **UI:** Tailwind CSS
- **Estado:** Context API (AuthContext, ThemeContext)
- **HTTP Client:** Axios con interceptores JWT

4.2 Flujo de Autenticación

1. Usuario ingresa credenciales en LoginScreen.jsx
2. Frontend POST a /api/v1/auth/login
3. Backend valida y retorna token JWT
4. Frontend almacena token en localStorage
5. Axios interceptor añade token a todas las peticiones
6. Backend valida token en endpoints protegidos

4.3 Comunicación API

- **Base URL:** http://localhost:8000/api/v1
- **Formato:** JSON

- **Headers:** Authorization: Bearer <token>
 - **Manejo de errores:** Códigos HTTP estándar (401, 403, 404, 500)
-

5. Instrucciones de Despliegue

Opción 1: Docker Compose (Recomendado)

Bash

```
docker-compose up -d
```

```
# Backend disponible en: http://localhost:8000
```

```
# Base de datos en: localhost:3306
```

Opción 2: Local con XAMPP

Bash

```
# 1. Iniciar MySQL en XAMPP
```

```
# 2. Crear base de datos 'condominio_db'
```

```
# 3. Ejecutar migraciones
```

```
cd backend
```

```
alembic upgrade head
```

```
# 4. Iniciar backend
```

```
uvicorn app.main:app --reload --host 0.0.0.0 --port 8000
```

```
# 5. Iniciar frontend
```

```
cd ../frontend
```

```
npm run dev
```

6. Conclusiones

El sistema cumple con todos los requisitos de la actividad:

- **Backend completo** con FastAPI y documentación automática
- **Autenticación segura** con JWT + bcrypt
- **Modelo relacional** robusto con 11 entidades
- **Migraciones** versionadas con Alembic
- **Dockerización** completa con Docker Compose
- **Integración** funcional frontend-backend
- **Sistema de roles** y permisos implementado

El proyecto demuestra dominio de arquitectura web moderna, mejores prácticas de seguridad y capacidad de integración full-stack.